Les tableaux

Extrait NSI 1re Ed. Ellipses

Exercice 69

On souhaite écrire un programme python permettant de travailler sur la pyramide des âges des français. Il fait intervenir un tableau `PDA = [691165, 710534, 728579, ...]` dans lequel la case d'index i contient **le nombre de français ayant l'âge** i.

```python
PDA = [691165, 710534, 728579, ..., 2160] # 106 valeurs

def examen_pda(tab_pda, age_min, age_max):
    """
    renvoie ...
    """
    n = 0
    for age in range(age_min, age_max):
        n = n + tab_pda[age]
    return n
```

1. Que fait la fonction `examen_pda` ?
2. Cette fonction n'est pas très robuste. En effet, si l'utilisateur l'appelle avec une valeur de `age_max` plus grande que 106 (*la taille du tableau* `PDA`), alors le programme va échouer suite à un accès en dehors des limites du tableau. Améliorer ce programme pour que l'utilisateur puisse spécifier une valeur de `age_max` arbitrairement grande, sans pour autant que le programme n'échoue.

```python
def examen_pda(tab_pda, age_min, age_max):
    """
    renvoie ...
    """
    # Supprimer 'pass' et compléter
    pass
```

Exercice 70

Écrire une fonction `occurrences(v, t)` qui renvoie le nombre d'occurrences de la valeur `v` dans le tableau `t`.

```python
def occurences(v, t):
    """
    renvoie le nombre d'occurence de v dans le tableau t
    v: entier
    t: tableau non vide
    """
    # Supprimer 'pass' et compléter
    pass
```

```python
# CELLULE DE TESTS - A DECOMMENTER PUIS EXECUTER
assert occurences(-1, [2,3,4,5,-1,6,7,-1]) == 2
print("Validé")
```

Exercice 72

Ecrire une fonction `max_tab(t)` qui prend un tableau `t` non vide en paramètre et qui renvoie l'élément maximum de ce tableau. **Interdiction d'utiliser la fonction native** `max` de python.

```python
def max_tab(t):
    """
    renvoie la valeur maximale rencontrée dans t
    t: tableau non vide
    """
    assert len(t) > 0, "Erreur: tableau vide"
    # Supprimer 'pass' et compléter
    pass
```

```python
# CELLULE DE TESTS - A DECOMMENTER PUIS EXECUTER
from random import randint
test = [randint(-10, 10) for i in range(20)]
assert max_tab(test) == max(test)
print("Validé")
```

Exercice 73

Écrire une fonction `hasard` qui tire au hasard mille entiers entre 1 et 10 et renvoie ensuite le nombre de fois que chaque nombre a été tiré. Appeler la fonction plusieurs fois. Conseil: penser à renvoyer un tableau.

```python
from random import randint


def hasard():
    """
    renvoie le nombre de tirage aléatoire de nombres
    compris 1 et 10
    """
    # Supprimer 'pass' et compléter
    pass
```

```python
# CELLULE DE TESTS - A DECOMMENTER PUIS EXECUTER
assert sum(hasard()) == 1000
print("OK")
```

Exercice 74

En mathématiques, la très célèbre *suite de Fibonacci* est une séquence infinie d'entiers définie de la façon suivante : on part des deux entiers 0 et 1 puis on construit à chaque fois l'entier suivant comme **la somme des deux entiers précédents**. 0, 1, 1, 2, 3, 5, ...

Écrire une fonction `fibo(k)` qui construit puis renvoie un tableau contenant les k premiers termes de la suite. Vérifier avec la cellule de test.

```python
def fibo(k):
    """
    renvoie les k premiers termes de la suite de fibonacci
    """
    # Supprimer 'pass' et compléter
    pass
```

```python
# CELLULE DE TESTS - A DECOMMENTER PUIS EXECUTER
assert fibo(30)[-1] == 514229
```

Exercice 75

Écrire une fonction `copie(t)` qui prend en paramètre un tableau `t` et renvoie une copie de ce tableau. Quelle expérience peut-on faire pour s'assurer qu'on ne s'est pas trompé ?

```python
def copie(t):
    """
    renvoie une copie de t;
    t: tableau
    """
    # Supprimer 'pass' et compléter
    pass
```

```python
# Comment être sûr que c'est une copie ?
t1 = [1, 2, 3, 4]
t2 = copie(t1)
t2[0] = -100 # On modifie le 1er élément de t2
print(f"t1 = {t1}")
print(f"t2 = {t2}")
```

Exercice 76

Écrire une fonction `ajout(v, t)` qui crée un nouveau tableau contenant d'abord tous les éléments de `t` puis `v` puis qui renvoie ce tableau.

```python
def ajout(v, t):
    """
    ajoute un élément à la copie d'un tableau et renvoie ce dernier
    t: tableau
    v: élément de type quelconque
    """
    # Supprimer 'pass' et compléter
    pass
```

```python
# CELLULE DE TESTS - A DECOMMENTER PUIS EXECUTER
assert ajout(1, []) == [1]
assert ajout(3, [1, 2]) == [1, 2, 3]
print("validé")
```

Exercice 77

Écrire une fonction `concatenation(t1, t2)` qui crée un nouveau tableau contenant, dans l'ordre, tous les éléments de `t1` puis tous les éléments de `t2`.

```python
def concatenation(t1, t2):
    """
    renvoie un tableau contenant la concaténation de t1 et t2
    t1, t2: tableau
    """
    # Supprimer 'pass' et compléter
    pass
```

```python
# CELLULE DE TESTS - A DECOMMENTER PUIS EXECUTER
assert concatenation([-1,3], [8,90]) == [-1, 3, 8, 90]
assert concatenation([], []) == []
print("Validé")
```

Exercice 78

Écrire une fonction `tableau_aleatoire(n, a, b)` qui renvoie un tableau de taille `n` contenant des entiers tirés au hasard entre `a` et `b`.

```
from random import randint


def tableau_aleatoire(n, a, b):
    """
    renvoie un tableau de taille n contenant des valeurs aléatoires
    n: entier positif
    a, b: entiers
    """
    # Supprimer 'pass' et compléter
    pass
```

Exercice 79

Écrire une fonction `tableau_croissant(n)` qui renvoie un tableau de taille `n` contenant des entiers tirés au hasard et ayant la propriété d'être trié par ordre croissant. Pour faire cela, on pourrait utiliser l'exercice précédent pour construire un tableau aléatoire, puis le trier avec un algorithme de tri (ce que nous verrons au cours de l'année en NSI).
Il y a néanmoins une façon plus simple de procéder, consistant à **remplir le tableau de gauche à droite en ajoutant à chaque fois un entier positif ou nul à l'élément précédent**. Ainsi, le tableau est trié par construction.

```
def tableau_croissant(n):
    """
    renvoie un tableau croissant de taille n;
    n: entier positif
    """
    # Supprimer 'pass' et compléter
    pass
```

```
tableau_croissant(8)
```

Exercice 80

Écrire une fonction `echange(tab, i, j)` qui échange dans le tableau `tab` les éléments aux indices `i` et `j`.

```python
def echange_tab(tab, i, j):
    """
    échange les éléments d'indice i et j d'un tableau;
    tab: tableau non vide
    i, j: entiers naturels
    """
    # Supprimer 'pass' et compléter
    pass
```

```python
t = [1,2,3,4,5,6,10]
echange_tab(t, 1, 6)
print(t)
```

Exercice 81

Écrire une fonction `somme(tab)` qui calcule et renvoie la somme des éléments d'un tableau d'entiers. En déduire une fonction `moyenne(tab)` qui calcule et renvoie la moyenne des éléments du tableau `tab`, supposé non vide. **Interdiction d'utiliser des fonctions natives de python**.

```python
def somme(tab):
    """
    renvoie la somme des éléments d'un tableau;
    tab: tableau
    """
    # Supprimer 'pass' et compléter
    pass

def moyenne(tab):
    """
    renvoie la moyenne des éléments d'un tableau
    tab: tableau
    """
    # Supprimer 'pass' et compléter
    pass
```

```python
moyenne([10, 12.5, 3, 7.5])
```

Exercice 82

Écrire une fonction `produit(tab)` qui calcule et renvoie le produit des éléments d'un tableau d'entiers. Si le tableau contient 0, la fonction devra renvoyer 0 sans terminer le calcul. **Interdiction d'utiliser des fonctions de bibliothèques python**

```python
def produit(tab):
    """
    renvoie le produit des éléments d'un tableau
    tab: tableau
    """
    # Supprimer 'pass' et compléter
    pass
```

```python
produit([1,2,3,4,5,6])
```